Assignment 1: Branch Predictor Implementation
COSC 6385 Computer Architecture
Daniel Prol - UH ID: 2446940
Repository: https://github.com/dprol/branch-predictor

Introduction

A branch predictor guesses the direction of a conditional branch, taken or not taken, before the branch is executed, so that the processor can keep fetching instructions. The Pentium M uses a hybrid predictor. A bimodal table of 2-bit counters, indexed with the branch address, predicts each branch from its own past outcomes. A global predictor, a small cache indexed with the branch address combined with the history of the previous branches, predicts the branches that depend on the path that led to them. When the global predictor has an entry for the branch (a hit), its prediction is used; otherwise the bimodal prediction is used.

In this assignment I have done the following:

1. Implemented the hybrid predictor (bimodal table and global predictor) in the class pm_predictor of src/my_predictor.h, with the sizes of the Pentium M in the course lecture slides (CompArch-Lec6-and-7-Bpred.pdf, slides 54 and 55): 4096 bimodal counters and a 4-way global predictor with 2048 entries.
2. Checked the implementation with the worked example of the assignment text (Appendix B, "Hybrid Predictor Example"). With the sizes and the allocation policy of the example, the class gives the same 22 predictions.
3. Ran the predictor on the 20 traces of the provided infrastructure (cbp2-infrastructure-v2). The average is 5.848 mispredictions per 1000 instructions (MPKI); the sample gshare predictor of the infrastructure gives 6.306.
4. Compared it with the bimodal table alone, with a 2-way global predictor and with the allocation policy of the worked example, to see what each part contributes.

What works

- Direction prediction of conditional branches with the bimodal table and the global predictor. A global hit overrides the bimodal prediction.
- The global predictor is a set-associative cache with valid bits, tags and LRU replacement. It has 4 ways (the assignment allows 2), and the sizes are parameters of the class.
- With the sizes and the allocation policy of the worked example of the assignment text, the class gives the same set, tag, global hit and final prediction as the 22 rows of the example.
- The average over the 20 traces is 5.848 MPKI. It is lower than the sample gshare in all 20 traces (6.306 on average), and the predictor uses less than half of the storage of gshare.

Limitations

- The loop predictor and the branch target prediction are not implemented; the assignment says that they can be skipped. target_prediction is always 0, and the extra credit class cpm_predictor is not changed.
- The PIR is not implemented; the assignment says that it is not required. The hash uses the global history register (the outcomes of the last 15 conditional branches) in its place,

as gshare does.

- The slides and the assignment text do not say when a new entry is allocated in the global predictor. I allocate one only after a misprediction (explained in "Update").

Submitted files

The code is submitted in branch-predictor.zip, together with this report. The same files are in the repository. The zip has the complete infrastructure with the traces, so it can be built and run as it is.

Path	Content
src/my_predictor.h	The predictor. pm_predictor is my implementation; gshare_predictor (the sample) and cpm_predictor (extra credit) are as provided.
src/test_example.cc (new)	Runs the worked example of the assignment text and compares each prediction with the table of the example.
src/compare.cc (new)	Runs the sample gshare, the bimodal table alone and three configurations of pm_predictor on one trace, in one pass, and prints the MPKI of each.
src/Makefile	Two new targets: test_example and compare.
run.sh, compare.sh (new)	Run predict or compare on all the traces and print the average. run.sh is the provided run script written in bash.
results.txt, comparison.txt	The output of ./run.sh traces and ./compare.sh traces.
changes.diff	Every line that I changed in the provided files (my_predictor.h and Makefile), as a unified diff.
README.md	How to build and run.
Other files	The provided infrastructure, not changed: the driver src/predict.cc and the other files of src/, the 20 traces in traces/ and the documentation in doc/.

This report does not copy the code. Each part is described in words, with the file, the function and the lines where it is. The line numbers are those of the submitted files.

Implementation

The provided src/my_predictor.h has three classes: gshare_predictor, a sample gshare predictor; pm_predictor, empty, which is the assignment; and cpm_predictor, empty, for the extra credit. The provided driver src/predict.cc creates a pm_predictor and, for every branch of the trace, calls predict before the branch and update after it, with the actual outcome. I wrote pm_predictor and its class pm_update; the other two classes and the driver are not changed. changes.diff shows the provided version and the new version of every changed line.

No external code was used. The set-associative lookup and the LRU replacement of the global predictor are written in pm_predictor.

Sizes and state (my_predictor.h)

Part	Where
pm_update: the values that predict saves for update (bimodal index, set, tag, hit way, conditional branch)	my_predictor.h, class pm_update (lines 64–71)
Default sizes: 4096 bimodal counters, 512 sets × 4 ways, 6-bit tag, 15-bit history	my_predictor.h, lines 75–79
Allocation policy of the global predictor (option alloc_on_any_miss)	my_predictor.h, lines 83–90
Tables: bimodal counters, and the counter, valid bit, tag and LRU age of each global entry	my_predictor.h, lines 92–97
The sizes are parameters of the constructor; initial values	my_predictor.h, pm_predictor constructor (lines 99–117)

The default sizes are the ones of the Pentium M in the lecture slides: 4096 bimodal counters indexed with IP[11:0] (slide 54), and a 4-way global predictor with 2048 entries, a 9-bit index and a 6-bit tag (slide 55). Because the sizes are parameters, the same class runs the small configuration of the worked example and the 2-way configuration of the comparison. All the counters start at 1 (weakly not taken) and all the global entries start invalid.

Storage of the submitted configuration:

Component	Bits
Bimodal table: 4096 × 2-bit counter	8,192
Global predictor: 2048 × (2-bit counter + 6-bit tag + 1 valid bit + 2 LRU bits)	22,528
Global history register	15
Total	30,735

The sample gshare has 32,768 2-bit counters (65,536 bits). In the simulator the LRU order is an age counter per entry; in hardware the order of 4 ways needs 2 bits per entry.

Prediction (my_predictor.h, predict)

Part	Where
Bimodal prediction: the low 12 bits of the branch address select a counter, and its upper bit is the prediction	my_predictor.h, predict (lines 133–135)
Global lookup: hash = address XOR history (15 bits); the upper 9 bits select the set and the lower 6 bits are the tag	my_predictor.h, predict (lines 137–140)
A valid entry of the set with the same tag is a hit, and its counter replaces the bimodal prediction	my_predictor.h, predict (lines 141–148)
Branches that are not conditional are predicted taken	my_predictor.h, predict (lines 150–152)
No target prediction	my_predictor.h, predict (lines 154–155)

The slides compute the hash with the PIR: HASH = PIR XOR IP, index = HASH[14:6] and tag = HASH[5:0] (slide 55). I use the global history register instead of the PIR, with the same split: the upper 9 bits of the hash select the set and the lower 6 bits are the tag. The worked example of the assignment text splits the hash in the same way (Row = upper bits, Tag = lower bits).

The ways of the set are searched in order, and the first valid way with the same tag is the hit. A tag is never stored twice in the same set, because a new entry is allocated only after a global miss.

Update (my_predictor.h, update)

Part	Where
Branches that are not conditional do not change the predictor	my_predictor.h, update (line 162)
Global hit: the counter of the entry is updated, and the entry becomes the most recently used of its set	my_predictor.h, update (lines 164–166)
Global miss and the branch was mispredicted: a new entry is allocated	my_predictor.h, update (lines 167–184)
Victim: the first invalid way of the set, otherwise the least recently used way	my_predictor.h, update (lines 168–176)
The counter of the new entry starts in the weak state of the outcome (2 if taken, 1 if not taken)	my_predictor.h, update (lines 179–182)
The bimodal counter and the history register are updated for every conditional branch	my_predictor.h, update (lines 185–188)
2-bit saturating counter	my_predictor.h, update_counter (lines 192–198)
LRU order: the used way gets age 0 and the other ways of the set get older	my_predictor.h, touch (lines 200–204)

The bimodal counter is updated also when the global predictor gave the prediction, so the bimodal table keeps learning every branch.

Allocation policy. The slides and the assignment text do not say when the global predictor allocates a new entry. The worked example allocates one on every global miss, and the new entry keeps the counter of the entry that it replaces. I allocate an entry only when the branch was mispredicted. If the bimodal table predicted the branch correctly, the branch does not need the global predictor for this history, and the entry is left for a branch that does. With this policy the average is 5.848 MPKI; with the policy of the example it is 7.756 (see Test 2). The policy of the example is still available as the option alloc_on_any_miss = true (lines 83–90, 167 and 179–180), and it is used to check the example.

Other changes (Makefile and scripts)

Change	Where
New targets test_example and compare, and clean also removes them	src/Makefile, lines 13–20
run.sh: the provided run script written in bash. The provided one needs csh and dc, which are not installed on many systems	run.sh (new)

Change	Where
compare.sh: runs compare on all the traces and prints the average of each predictor	compare.sh (new)
The provided scripts run and run_extra had no execute permission; I added it	run, run_extra

Results and Test cases

I checked the predictor in two ways: with the worked example of the assignment text, where every prediction is known, and with the 20 traces of the infrastructure, where the result is the number of mispredictions.

Test	What it tests
Test 1: worked example	src/test_example.cc. The 22 branches of Appendix B of the assignment text, with the sizes and the initial counters of the example. Each prediction is compared with the table of the example.
Test 2: traces	The 20 traces of the infrastructure (100 million instructions each), with the provided driver predict.cc (run.sh) and with compare.cc, which runs 5 predictors on each trace.

Test 1: worked example of the assignment text

The example uses 8-bit branch addresses, 16 bimodal counters, a 4-bit GHR and a global predictor of 2 ways and 4 sets with 2-bit tags. The 4 address bits used for the indices are bits 5 to 2, so b1 (0x44) has index 1, b2 (0x6C) index 11 and b3 (0x90) index 4, as in the Local Predictor table of the example.

Part of the test	Where
The 22 branches with their outcomes, and the final prediction and global hit of the example	test_example.cc, lines 15–22
pm_predictor with the sizes of the example: 4-bit bimodal index, 4 sets, 2-bit tag, 2 ways, 4-bit history, address shifted right by 2 bits, and allocation on every global miss	test_example.cc, line 24
Initial counters and LRU state of the example	test_example.cc, lines 26–32
For each branch: predict, compare with the example, update with the outcome	test_example.cc, lines 35–48

The result is 0 mismatches. The table shows the output of test_example. The set and the tag are also the same as the Row and Tag columns of the example.

#	Branch	Actual	Set	Tag	Global	Prediction	Example
1	b1	T	0	01	no hit	NT	NT
2	b3	T	1	01	no hit	NT	NT
3	b1	NT	0	10	no hit	T	T
4	b2	NT	3	01	no hit	T	T
5	b3	T	2	00	no hit	NT	NT
6	b1	T	2	00	hit	T	T
7	b3	T	1	11	no hit	T	T
8	b1	NT	1	10	no hit	T	T
9	b2	NT	1	01	no hit	T	T
10	b3	T	2	00	hit	T	T
11	b1	NT	2	00	hit	T	T
12	b2	T	2	01	no hit	NT	NT
13	b1	NT	1	00	no hit	NT	NT
14	b2	NT	0	01	hit	NT	NT
15	b3	T	0	00	no hit	T	T
16	b1	T	2	00	hit	T	T
17	b3	T	1	11	no hit	T	T
18	b1	NT	1	10	no hit	NT	NT
19	b2	NT	1	01	no hit	NT	NT
20	b3	T	2	00	hit	T	T
21	b1	T	2	00	hit	T	T
22	b3	T	1	11	no hit	T	T

Prediction is the final prediction of the simulator and Example is the Final Prediction column of the example.

Test 2: the 20 traces

MPKI is the number of direction mispredictions of conditional branches per 1000 instructions; each trace has 100 million instructions. This is the result that the provided driver predict.cc prints. The columns of the table are:

- gshare: the sample predictor of the infrastructure (32,768 counters, 15 bits of history).
- Bimodal: the 4096-entry bimodal table alone.
- PM any miss: pm_predictor with 4 ways and the allocation of the worked example (a new entry on every global miss).
- PM 2-way: pm_predictor with a 2-way global predictor of the same size (1024 sets, 5-bit tag), with allocation after a misprediction.
- PM 4-way: the submitted predictor (the default sizes of pm_predictor).

The five predictors are created in compare.cc (lines 57–63); the bimodal table alone is the class bimodal_predictor (compare.cc, lines 15–46). The PM 4-way column is the same as the output

of the provided driver for all 20 traces (results.txt).

Trace	gshare	Bimodal	PM any miss	PM 2-way	PM 4-way
164.gzip	12.473	15.030	12.690	12.458	12.436
175.vpr	13.415	14.671	15.965	13.519	13.298
176.gcc	11.254	19.347	18.006	9.534	9.285
181.mcf	15.837	33.245	18.264	15.927	15.621
186.crafty	5.837	13.075	9.101	5.351	5.211
197.parser	10.008	15.417	10.929	8.656	8.538
201.compress	7.831	9.356	8.014	7.748	7.702
202.jess	1.562	8.809	2.001	1.134	1.118
205.raytrace	2.756	3.806	3.606	2.421	2.307
209.db	3.909	6.480	4.818	3.665	3.615
213.javac	2.267	3.181	2.996	2.061	2.037
222.mpegaudio	2.188	3.154	2.927	2.057	2.028
227.mtrt	2.657	3.706	3.439	2.419	2.383
228.jack	3.033	7.691	4.234	2.385	2.353
252.eon	1.807	9.405	2.145	1.470	1.425
253.perlbmk	2.554	5.955	5.120	1.864	1.819
254.gap	3.926	9.675	4.304	3.530	3.507
255.vortex	1.222	1.916	1.636	0.836	0.828
256.bzip2	0.094	0.112	0.110	0.058	0.057
300.twolf	21.489	21.942	24.816	21.529	21.389
Average	**6.306**	**10.299**	**7.756**	**5.931**	**5.848**

- The global predictor is what makes the hybrid work: the bimodal table alone has 10.299 MPKI and the hybrid 5.848, 43% less.
- The submitted predictor is better than the sample gshare in all 20 traces: 7% fewer mispredictions on average, with less than half of the storage. The largest difference is in 176.gcc (11.254 to 9.285). I did not measure the cause. A probable one is that the tags avoid the aliasing between branches that gshare has, and that the bimodal table predicts the branches that do not need history, so they do not use global entries.
- The allocation policy is the most important decision. With the allocation of the worked example the average is 7.756, worse than gshare, and in 175.vpr and 300.twolf it is worse than the bimodal table alone. Every global miss replaces an entry, so the entries of the branches that need history are evicted by branches that the bimodal table already predicts well. Allocating only after a misprediction gives 5.848.
- With the same 2048 entries, 4 ways are a little better than 2 ways (5.848 against 5.931).
- 164.gzip, 175.vpr, 181.mcf and 300.twolf stay above 12 MPKI with all the predictors, and the global predictor helps little there (300.twolf: 21.942 with the bimodal table alone, 21.389 with the hybrid).

How to run

```
unzip branch-predictor.zip
(or: git clone https://github.com/dprol/branch-predictor.git)
cd branch-predictor/src
make                                    # builds predict
make test_example compare
./test_example                          # worked example, prints "0 mismatches"
cd ..
./run.sh traces                         # pm_predictor on the 20 traces
./compare.sh traces                     # the 5 predictors on the 20 traces
```

g++, make and bzip2 are required (the traces are read with /usr/bin/bzip2, see src/trace.h). The provided ./run traces does the same as ./run.sh traces, but it needs csh and dc.